

```

# =====
# 1. INSTALL (Colab only)
# =====
!pip install transformers datasets -q

# =====
# 2. IMPORTS
# =====
import pandas as pd
import torch
from datasets import Dataset
from transformers import (
    T5Tokenizer,
    T5ForConditionalGeneration,
    DataCollatorForSeq2Seq
)
from torch.utils.data import DataLoader

# =====
# 3. LOAD DATA
# =====
df = pd.read_csv("BankFAQs.csv")

# Fix column names
df = df.rename(columns={
    "Question": "question",
    "Answer": "answer"
})

df = df[["question", "answer"]].dropna().drop_duplicates()

# =====
# 4. DATASET
# =====
dataset = Dataset.from_pandas(df)
dataset = dataset.train_test_split(test_size=0.3, seed=42)

train_data = dataset["train"]
test_data = dataset["test"]

# =====
# 5. MODEL
# =====
model_name = "t5-small"

tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# =====
# 6. TOKENIZATION (FIXED)
# =====
def tokenize_function(example):
    inputs = ["question: " + q for q in example["question"]]

    model_inputs = tokenizer(
        inputs,
        max_length=128,
        truncation=True,
        padding="max_length"
    )

```

```

        padding= max_length
    )

    labels = tokenizer(
        text_target=example["answer"],
        max_length=128,
        truncation=True,
        padding="max_length"
    )

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

train_tokenized = train_data.map(tokenize_function, batched=True)
test_tokenized = test_data.map(tokenize_function, batched=True)

# Remove raw text
train_tokenized = train_tokenized.remove_columns(["question", "answer"])
test_tokenized = test_tokenized.remove_columns(["question", "answer"])

# =====
# 7. MASK PADDING TOKENS
# =====
def mask_labels(example):
    example["labels"] = [
        (1 if l != tokenizer.pad_token_id else -100)
        for l in example["labels"]
    ]
    return example

train_tokenized = train_tokenized.map(mask_labels)
test_tokenized = test_tokenized.map(mask_labels)

# =====
# 8. FORMAT + DATALOADER
# =====
train_tokenized.set_format(type="torch")

data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model
)

train_loader = DataLoader(
    train_tokenized,
    batch_size=8,
    shuffle=True,
    collate_fn=data_collator
)

# =====
# 9. TRAINING
# =====
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)

model.train()
print("=== TRAINING STARTED ===")

for epoch in range(5):
    total_loss = 0

    for batch in train_loader:
        batch = {k: v.to(device) for k, v in batch.items()}

```

```
optimizer.zero_grad()

outputs = model(
    input_ids=batch["input_ids"],
    attention_mask=batch["attention_mask"],
    labels=batch["labels"]
)

loss = outputs.loss
loss.backward()
optimizer.step()

total_loss += loss.item()

print(f"Epoch {epoch+1}: Loss = {total_loss/len(train_loader):.4f}")

# =====
# 10. INFERENCE (FIXED)
# =====
def generate_answer(question):
    model.eval()

    input_text = "question: " + question

    inputs = tokenizer(
        input_text,
        return_tensors="pt",
        truncation=True,
        padding=True
    ).to(device)

    outputs = model.generate(
        input_ids=inputs["input_ids"],
        attention_mask=inputs["attention_mask"],
        max_length=100,
        num_beams=5,
        early_stopping=True,
        no_repeat_ngram_size=2
    )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# =====
# 11. TEST
# =====
print("\n=== TESTING ===")

q = "How to open a bank account?"
print("Q:", q)
print("A:", generate_answer(q))
```

```
Loading weights: 100% 131/131 [00:00<00:00, 604.66it/s, Materializing param=shared.weight]
Map: 100% 1049/1049 [00:00<00:00, 3337.34 examples/s]
Map: 100% 450/450 [00:00<00:00, 3004.45 examples/s]
Map: 100% 1049/1049 [00:00<00:00, 2394.62 examples/s]
Map: 100% 450/450 [00:00<00:00, 2157.61 examples/s]
=== TRAINING STARTED ===
/usr/local/lib/python3.12/dist-packages/transformers/data/data_collator.py:600: UserWarning: Creating
  batch["labels"] = torch.tensor(batch["labels"], dtype=torch.int64)
Epoch 1: Loss = 4.4919
Epoch 2: Loss = 4.0911
Epoch 3: Loss = 3.9616
Epoch 4: Loss = 3.8522
Epoch 5: Loss = 3.7829

=== TESTING ===
Q: How to open a bank account?
A: You can open a bank account on your own.
```